PROGRAM

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <time.h>

#define MAX_USERS 50
#define MAX_TRAINS 20
#define MAX_BOOKINGS 500
#define MIN_BALANCE 2000
#define ADMIN_PASSWORD "jdg"

typedef struct { char name[20]; char pass[20]; double balance; } User;
typedef struct {
  int id; char name[30]; char from[20]; char to[20];
  int date;         /* YYYYMMDD */
  int time;         /* HHMM     */
  int km;
  int seats[2];     /* [0]=Sleeper, [1]=AC */
} Train;
typedef struct {
  int pnr; char user[20]; int trainId; int cls; int pax;
  double fare; int active;
} Booking;

User users[MAX_USERS];         int nUsers;
Train trains[MAX_TRAINS];      int nTrains;
Booking bookings[MAX_BOOKINGS]; int nBookings;
int cur = -1;                  /* index of logged-in user */
const char *CLS[2] = {"Sleeper", "AC"};
const double RATE[2] = {1.0, 2.0};   /* Rs per km per passenger */

/* ---------- input helpers ---------- */
void readLine(const char *prompt, char *buf, int n) {
  printf("%s", prompt);
  if (!fgets(buf, n, stdin)) exit(0);
  buf[strcspn(buf, "\n")] = '\0';
}
int readInt(const char *prompt) {
  char b[32]; readLine(prompt, b, sizeof b); return atoi(b);
}
double readMoney(const char *prompt) {
  char b[32]; readLine(prompt, b, sizeof b); return atof(b);
}
int same(const char *a, const char *b) {        /* case-insensitive == */
  while (*a && *b && tolower((unsigned char)*a) == tolower((unsigned char)*b)) a++, b++;
  return *a == '\0' && *b == '\0';
}

/* ---------- file storage ---------- */
void save(void) {
```

```
  FILE *f;
  if ((f = fopen("users.dat", "wb")))    { fwrite(users, sizeof(User), nUsers, f);       fclose(f); }
  if ((f = fopen("trains.dat", "wb")))   { fwrite(trains, sizeof(Train), nTrains, f);    fclose(f); }
  if ((f = fopen("bookings.dat", "wb"))) { fwrite(bookings, sizeof(Booking), nBookings, f); fclose(f); }
}
void load(void) {
  FILE *f;
  if ((f = fopen("users.dat", "rb")))    { nUsers = fread(users, sizeof(User), MAX_USERS, f);         fclose(f); }
  if ((f = fopen("trains.dat", "rb")))   { nTrains = fread(trains, sizeof(Train), MAX_TRAINS, f);     fclose(f); }
  if ((f = fopen("bookings.dat", "rb"))) { nBookings = fread(bookings, sizeof(Booking), MAX_BOOKINGS, f); fclose(f); }
}

/* ---------- lookups ---------- */
int findUser(const char *name) {
  for (int i = 0; i < nUsers; i++) if (strcmp(users[i].name, name) == 0) return i;
  return -1;
}
int findTrain(int id) {
  for (int i = 0; i < nTrains; i++) if (trains[i].id == id) return i;
  return -1;
}
int findBooking(int pnr) {
  for (int i = 0; i < nBookings; i++)
    if (bookings[i].pnr == pnr && strcmp(bookings[i].user, users[cur].name) == 0) return i;
  return -1;
}

/* ---------- accounts ---------- */
void registerUser(void) {
  if (nUsers >= MAX_USERS) { printf("User limit reached.\n"); return; }
  User u;
  readLine("Username: ", u.name, sizeof u.name);
  if (u.name[0] == '\0' || findUser(u.name) >= 0) { printf("Invalid or duplicate username.\n"); return; }
  readLine("Password: ", u.pass, sizeof u.pass);
  do {
    u.balance = readMoney("Initial wallet balance (min 2000): ");
  } while (u.balance < MIN_BALANCE);
  users[nUsers++] = u;
  save();
  printf("Registration successful.\n");
}

int loginUser(void) {
  char name[20], pass[20];
  readLine("Username: ", name, sizeof name);
  readLine("Password: ", pass, sizeof pass);
  int i = findUser(name);
  if (i >= 0 && strcmp(users[i].pass, pass) == 0) { cur = i; printf("Login successful.\n"); return 1; }
  printf("Invalid username or password.\n");
  return 0;
}

void wallet(void) {
```

```
  printf("Wallet balance: Rs %.2f\n", users[cur].balance);
  double d = readMoney("Deposit amount (0 to skip): ");
  if (d > 0) { users[cur].balance += d; save(); printf("New balance: Rs %.2f\n", users[cur].balance); }
}

/* ---------- admin ---------- */
void addTrain(void) {
  if (nTrains >= MAX_TRAINS) { printf("Train limit reached.\n"); return; }
  Train t;
  t.id = readInt("Train ID: ");
  if (findTrain(t.id) >= 0) { printf("Train ID already exists.\n"); return; }
  readLine("Train name: ", t.name, sizeof t.name);
  readLine("From: ", t.from, sizeof t.from);
  readLine("To: ", t.to, sizeof t.to);
  t.date = readInt("Date (YYYYMMDD): ");
  t.time = readInt("Departure time (HHMM): ");
  t.km = readInt("Distance (km): ");
  t.seats[0] = readInt("Sleeper seats: ");
  t.seats[1] = readInt("AC seats: ");
  trains[nTrains++] = t;
  save();
  printf("Train added.\n");
}

void listTrains(void) {
  printf("%-5s %-16s %-10s %-10s %-9s %-5s %-4s %s\n",
        "ID", "Name", "From", "To", "Date", "Time", "Km", "Seats(SL/AC)");
  for (int i = 0; i < nTrains; i++)
    printf("%-5d %-16s %-10s %-10s %-9d %04d  %-4d %d/%d\n", trains[i].id, trains[i].name,
          trains[i].from, trains[i].to, trains[i].date, trains[i].time, trains[i].km,
          trains[i].seats[0], trains[i].seats[1]);
}

void listBookings(void) {
  printf("%-7s %-10s %-6s %-8s %-4s %-9s %s\n", "PNR", "User", "Train", "Class", "Pax", "Fare", "Status");
  for (int i = 0; i < nBookings; i++)
    printf("%-7d %-10s %-6d %-8s %-4d %-9.2f %s\n", bookings[i].pnr, bookings[i].user,
          bookings[i].trainId, CLS[bookings[i].cls], bookings[i].pax, bookings[i].fare,
          bookings[i].active ? "CONFIRMED" : "CANCELLED");
}

void viewFeedback(void) {
  FILE *f = fopen("feedback.txt", "r");
  char line[200]; int n = 0, sum = 0;
  if (!f) { printf("No feedback yet.\n"); return; }
  while (fgets(line, sizeof line, f)) {
    printf("%s", line);
    sum += line[0] - '0'; n++;
  }
  fclose(f);
  if (n) printf("Average rating: %.1f / 5 (%d review(s))\n", (double)sum / n, n);
}
```

```
void adminMenu(void) {
  int c;
  do {
    printf("\n--- Admin Menu ---\n1. Add train\n2. List trains\n3. All bookings\n4. Feedback\n5. Back\n");
    c = readInt("Choice: ");
    if (c == 1) addTrain();
    else if (c == 2) listTrains();
    else if (c == 3) listBookings();
    else if (c == 4) viewFeedback();
  } while (c != 5);
}

/* ---------- booking ---------- */
void searchTrains(void) {
  char from[20], to[20];
  readLine("From: ", from, sizeof from);
  readLine("To: ", to, sizeof to);
  int date = readInt("Date (YYYYMMDD): "), found = 0;
  for (int i = 0; i < nTrains; i++)
    if (same(trains[i].from, from) && same(trains[i].to, to) && trains[i].date == date) {
      printf("Train %d  %s  dep %04d  %d km  seats SL=%d AC=%d\n", trains[i].id, trains[i].name,
             trains[i].time, trains[i].km, trains[i].seats[0], trains[i].seats[1]);
      found = 1;
    }
  if (!found) printf("No trains found.\n");
}

void bookTicket(void) {
  int t = findTrain(readInt("Enter train ID: "));
  if (t < 0) { printf("Train not found.\n"); return; }
  int cls = readInt("Class (1=Sleeper, 2=AC): ") - 1;
  int pax = readInt("Number of passengers (1-6): ");
  if (cls < 0 || cls > 1 || pax < 1 || pax > 6) { printf("Invalid input.\n"); return; }
  if (trains[t].seats[cls] < pax) { printf("Only %d seat(s) left.\n", trains[t].seats[cls]); return; }
  if (nBookings >= MAX_BOOKINGS) { printf("Booking limit reached.\n"); return; }

  double fare = trains[t].km * RATE[cls] * pax;
  printf("Fare: %d km x Rs %.1f x %d = Rs %.2f\n", trains[t].km, RATE[cls], pax, fare);
  if (users[cur].balance < fare) { printf("Insufficient balance. Deposit money in the wallet.\n"); return; }

  users[cur].balance -= fare;
  trains[t].seats[cls] -= pax;
  Booking b = { 100001 + nBookings, "", trains[t].id, cls, pax, fare, 1 };
  strcpy(b.user, users[cur].name);
  bookings[nBookings++] = b;
  save();
  printf("Booking CONFIRMED. PNR: %d\nRemaining balance: Rs %.2f\n", b.pnr, users[cur].balance);
}

void checkStatus(void) {
  int b = findBooking(readInt("Enter PNR: "));
  if (b < 0) { printf("PNR not found.\n"); return; }
  int t = findTrain(bookings[b].trainId);
```

```
  printf("PNR %d | %s | %s -> %s | %d | %s x%d | Rs %.2f | %s\n", bookings[b].pnr,
         trains[t].name, trains[t].from, trains[t].to, trains[t].date,
         CLS[bookings[b].cls], bookings[b].pax, bookings[b].fare,
         bookings[b].active ? "CONFIRMED" : "CANCELLED");
}

/* 1 if departure is at least 12 hours away */
int moreThan12Hours(const Train *t) {
  struct tm d = {0};
  d.tm_year = t->date / 10000 - 1900;
  d.tm_mon = (t->date / 100) % 100 - 1;
  d.tm_mday = t->date % 100;
  d.tm_hour = t->time / 100;
  d.tm_min = t->time % 100;
  d.tm_isdst = -1;
  return difftime(mktime(&d), time(NULL)) >= 12 * 3600;
}

void cancelTicket(void) {
  int b = findBooking(readInt("Enter PNR to cancel: "));
  if (b < 0 || !bookings[b].active) { printf("No active booking with that PNR.\n"); return; }
  int t = findTrain(bookings[b].trainId);
  int full = moreThan12Hours(&trains[t]);
  double refund = full ? bookings[b].fare : bookings[b].fare * 0.6;
  bookings[b].active = 0;
  trains[t].seats[bookings[b].cls] += bookings[b].pax;
  users[cur].balance += refund;
  save();
  printf("Ticket cancelled. %s refund of Rs %.2f credited.\nBalance: Rs %.2f\n",
         full ? "100%" : "60%", refund, users[cur].balance);
}

void feedback(void) {
  int r = readInt("Rate our service (1-5): ");
  char c[150];
  if (r < 1 || r > 5) { printf("Invalid rating.\n"); return; }
  readLine("Comment: ", c, sizeof c);
  FILE *f = fopen("feedback.txt", "a");
  if (f) { fprintf(f, "%d | %s | %s\n", r, users[cur].name, c); fclose(f); }
  printf("Thank you for your feedback!\n");
}

void userMenu(void) {
  int c;
  do {
    printf("\n--- User Menu ---\n1. Search trains\n2. Book ticket\n3. Check status\n"
           "4. Cancel ticket\n5. Wallet\n6. Feedback\n7. Logout\n");
    c = readInt("Choice: ");
    if (c == 1) searchTrains();
    else if (c == 2) bookTicket();
    else if (c == 3) checkStatus();
    else if (c == 4) cancelTicket();
    else if (c == 5) wallet();
```

```
    else if (c == 6) feedback();
  } while (c != 7);
  cur = -1;
}

int main(void) {
  load();
  int c;
  do {
    printf("\n=== RAILWAY RESERVATION SYSTEM ===\n1. Register\n2. Login\n3. Admin login\n4. Exit\n");
    c = readInt("Choice: ");
    if (c == 1) registerUser();
    else if (c == 2 && loginUser()) userMenu();
    else if (c == 3) {
      char p[20];
      readLine("Admin password: ", p, sizeof p);
      if (strcmp(p, ADMIN_PASSWORD) == 0) adminMenu(); else printf("Wrong password.\n");
    }
  } while (c != 4);
  save();
  printf("Goodbye!\n");
  return 0;
}
```

OUTPUT

```
=== RAILWAY RESERVATION SYSTEM ===
1. Register
2. Login
3. Admin login
4. Exit
Choice: 1
Username: asha
Password: abc
Initial wallet balance (min 2000): 5000
Registration successful.

Choice: 3
Admin password: jdg

--- Admin Menu ---
1. Add train
2. List trains
3. All bookings
4. Feedback
5. Back
Choice: 1
Train ID: 101
Train name: Kovai Express
From: Coimbatore
To: Salem
Date (YYYYMMDD): 20261115
Departure time (HHMM): 0600
Distance (km): 160
Sleeper seats: 100
AC seats: 50
Train added.

[menu]
Choice: 1
Train ID: 102
Train name: Evening Fast
From: Coimbatore
To: Salem
Date (YYYYMMDD): 20261031
Departure time (HHMM): 1800
Distance (km): 160
Sleeper seats: 100
AC seats: 50
Train added.

[menu]
Choice: 2
ID   Name           From       To         Date      Time  Km   Seats(SL/AC)
101  Kovai Express  Coimbatore Salem      20261115  0600  160  100/50
102  Evening Fast   Coimbatore Salem      20261031  1800  160  100/50

[menu]
Choice: 5

Choice: 2
Username: asha
Password: abc
Login successful.

--- User Menu ---
1. Search trains
2. Book ticket
3. Check status
4. Cancel ticket
```

```
5. Wallet
6. Feedback
7. Logout
Choice: 1
From: Coimbatore
To: Salem
Date (YYYYMMDD): 20261115
Train 101  Kovai Express  dep 0600  160 km  seats SL=100 AC=50

[menu]
Choice: 2
Enter train ID: 101
Class (1=Sleeper, 2=AC): 2
Number of passengers (1-6): 2
Fare: 160 km x Rs 2.0 x 2 = Rs 640.00
Booking CONFIRMED. PNR: 100001
Remaining balance: Rs 4360.00

[menu]
Choice: 3
Enter PNR: 100001
PNR 100001 | Kovai Express | Coimbatore -> Salem | 20261115 | AC x2 | Rs 640.00 | CONFIRMED

[menu]
Choice: 5
Wallet balance: Rs 4360.00
Deposit amount (0 to skip): 500
New balance: Rs 4860.00

[menu]
Choice: 6
Rate our service (1-5): 5
Comment: Easy and fast booking
Thank you for your feedback!

[menu]
Choice: 7

Choice: 3
Admin password: jdg

[admin menu]
Choice: 3
PNR     User       Train  Class    Pax  Fare      Status
100001  asha       101    AC       2    640.00    CANCELLED
100002  asha       102    Sleeper  1    160.00    CANCELLED

[admin menu]
Choice: 4
5 | asha | Easy and fast booking
Average rating: 5.0 / 5 (1 review(s))

[admin menu]
Choice: 5

=== RAILWAY RESERVATION SYSTEM ===
Choice: 4
Goodbye!
```